Frontend Live Data Behavior

This dashboard is designed to look and behave like an operator-facing microgrid EMS console. It can work in two modes:

1. **Live API mode**
2. **Cached snapshot mode**

1. Live API Mode

Live API mode is active when the FastAPI backend is running:

```
cd "C:\Users\ayush\Desktop\Final Year Project"
venv\Scripts\python.exe -m uvicorn backend.app.main:app --host 127.0.0.1 --port
8000
```

Then start the frontend:

```
cd "C:\Users\ayush\Desktop\Final Year Project\frontend"
npm.cmd run dev
```

Open:

```
http://127.0.0.1:5173
```

When live API mode is working, the dashboard status line shows something like:

```
Status: online | Polling every 30s | Last update ... | API http://127.0.0.1:8000
```

Update Frequency

The frontend refreshes automatically:

```
Immediately on page load
Every 30 seconds in normal dataset-backed mode
Every 1.5 seconds while the live simulator is running
Whenever the Refresh button is clicked
Whenever a scenario simulation is run
```

The polling interval is defined in:

```
frontend/src/App.jsx
```

Current normal-mode code:

```
window.setInterval(refresh, 30000);
```

That means the frontend asks the backend for new EMS data every **30 seconds** in normal mode. When the operator clicks **Start live stream**, the backend starts a synthetic telemetry generator and the frontend switches to a **1.5-second** refresh cadence.

Synthetic Live Stream

The dashboard now includes a **Start live stream** button. It calls:

```
POST /live/start
```

The backend service then starts generating realistic live telemetry:

```
solar_kw
load_kw
tariff_inr_kwh
battery_soc_pct
battery_power_kw
grid_kw
load_shed_kw
operator_action
override_mode
weather fields
```

The generator runs in:

```
backend/app/services/live_simulator.py
```

Runtime telemetry is also written to:

```
data/runtime/live_telemetry.jsonl
```

The operator can stop it:

```
POST /live/stop
```

The dashboard also exposes override modes:

```
auto
force_charge
force_discharge
island
```

These call:

```
POST /live/override
```

What Updates Every 30 Seconds

Every polling cycle calls these backend endpoints:

```
GET /forecast
GET /optimize
GET /metrics
GET /alerts
```

These update the following dashboard sections:

Dashboard section	API endpoint	What can change
Live Energy Dashboard	`/optimize`	solar kW, load kW, battery SoC, grid kW
Forecast Visualization	`/forecast`	solar forecast curve, load forecast curve
Smart Decision Panel	`/optimize`	charge, discharge, or hold recommendation
Dispatch Table	`/optimize` and `/decisions`	action plan and reasons
Alerts Panel	`/alerts`	peak demand risk, renewable drop, battery warnings
Cost Analytics	`/metrics`	baseline cost, optimized cost, savings
Sustainability Metrics	`/metrics`	renewable share, grid dependency, self-sufficiency

The scenario simulator updates only when the operator clicks **Run**:

```
POST /simulate
```

Important: What “Live” Means In This Project

Right now, this project is **not connected to real SCADA, smart meters, inverter telemetry, or a live BMS**.

The live API is live in the software sense:

```
Frontend polling
→ FastAPI backend
→ EMS dataset / forecast files / dispatch engine
→ updated JSON response
→ dashboard refresh
```

The current data source is:

```
NASA POWER CSV
→ cleaned weather and solar data
→ realistic synthetic load/tariff/battery data
→ LSTM forecast outputs
→ dispatch and metrics engine
```

Main processed EMS file:

```
data/processed/ems_dataset.csv
```

Because this is dataset-backed, repeated 30-second refreshes may show the same values unless one of these changes:

```
data/processed/ems_dataset.csv is regenerated
forecast files are regenerated
the backend is connected to a live data source
a scenario simulation is run
frontend cache files are regenerated
```

So:

```
Does the frontend poll live? Yes, every 30 seconds.
Does the displayed data always change every 30 seconds? Not unless the backend
data changes.
```

Cached Snapshot Mode

If the backend is not reachable, the dashboard falls back to cached JSON files:

```
frontend/public/api-cache/forecast.json
frontend/public/api-cache/optimize.json
frontend/public/api-cache/metrics.json
frontend/public/api-cache/alerts.json
```

In that mode, the dashboard status line shows:

```
offline: using cached EMS snapshot
```

Cached mode is useful because the dashboard remains populated for demos, reviews, and offline explanation. However, cached mode is static. It does not update until the cache is regenerated.

Regenerate the cache:

```
cd "C:\Users\ayush\Desktop\Final Year Project"
venv\Scripts\python.exe backend\scripts\export_frontend_cache.py
```

What Time Is Shown?

The timestamps shown in the dashboard come from the EMS dataset and forecast artifacts, not from the current wall-clock time.

The NASA POWER source data covers:

```
2020-01-01 through 2025-12-31 UTC
```

The backend converts timestamps into local operator time:

```
Asia/Kolkata
```

That means dashboard timestamps can show historical or forecast-derived times such as late 2025 or early 2026. This is expected for the current dataset-backed system.

If this is connected to real telemetry later, the same API structure can return current wall-clock timestamps from:

```
smart meters
inverter telemetry
BMS telemetry
SCADA historian
MQTT stream
OPC UA server
MATLAB/Simulink real-time output
```

What Data Can Change In Real Deployment

In a real connected microgrid, every 30-second poll can update:

```
PV generation
load demand
battery SoC
battery charge/discharge power
grid import/export
tariff period
dispatch action
cost estimate
renewable percentage
grid dependency
alerts
forecast horizon
```

Typical practical update rates:

Data type	Practical update rate
Smart meter power	1 to 30 seconds
Battery SoC	5 to 60 seconds
Inverter/PV power	1 to 10 seconds
Grid import/export	1 to 30 seconds
Forecast refresh	5 to 60 minutes
Tariff schedule	hourly or day-ahead
Operator dashboard polling	30 seconds in this project

How To Make This Truly Live

To make the values change from real field data, replace the dataset-backed loader in:

```
backend/app/services/repository.py
```

Current behavior:

```
read data/processed/ems_dataset.csv
return a recent operating window
```

Real deployment behavior:

```
read latest telemetry from SCADA / MQTT / database / inverter API / smart meter
API
append it to the EMS store
run forecasting and dispatch
serve the latest result to the frontend
```

The frontend does not need a major redesign for that. It already polls the backend and re-renders the operator panels from API responses.